

PROJECT CONTEXT DOCUMENT

CEYPETCO Website Redesign

Tech stack, architecture, file structure, runtime conventions, and guidance for an AI coding assistant.

Repository: ceypetco-redesign

Generated from: the repository contents on 7 October 2026

Purpose: upload this PDF to ChatGPT before asking it to explain, debug, review, or modify this project.

1. Project summary

This repository is a full-stack rebuild of the official CEYPETCO website. It contains a public-facing content site and a protected administration/CMS interface. The frontend and backend are separate applications in one repository.

Architecture: browser → React single-page application → REST API under `/api` → Express controllers/models → MySQL. Uploaded images and documents are stored locally by the backend and served from `/uploads/images` and `/uploads/docs`.

Area	Technology	Observed version / role
Language	JavaScript	JSX + ES modules in frontend; CommonJS in backend
Runtime	Node.js	22+ (frontend specifies 22.12+)
Frontend	React, React DOM	19.2.x single-page application
Build tooling	Vite + React plugin	Vite 8.2.x
Styling	Tailwind CSS + custom CSS	Tailwind 4.3.x via Vite plugin
Routing	React Router	7.18.x; public catch-all plus protected admin routes
HTTP/UI helpers	Axios, DOMPurify, Lucide React, React Hot Toast, Recharts	API calls, HTML sanitization, icons, notifications, and charts
Backend	Express	5.2.x REST API
Database	MySQL	<code>mysql2</code> / <code>promise</code> connection pool
Authentication	JWT + <code>bcryptjs</code>	Token authentication, password hashing, and role checks
Uploads	Multer + local filesystem	Images and documents under <code>backend/uploads</code>
Security/ops	Helmet, CORS, rate limiting, Morgan, dotenv	Headers, origin control, throttling, logging, environment config
Deployment	Plesk; Vercel config also present	Separate frontend build and Node API; root backend workspace fallback

2. Application structure

Frontend

- `frontend/src/main.jsx` creates the React root and wires the error boundary, browser router, authentication provider, language provider, and toast notifications.

- `frontend/src/routes/AppRoutes.jsx` defines login, protected admin routes, role-restricted routes, and a catch-all public site route.
- `frontend/src/App.jsx` is the large public-site application entry and includes many public page sections and default content.
- `frontend/src/admin` contains the dashboard layout, shared admin components, and CRUD management pages.
- `frontend/src/services` and `frontend/src/api.js` centralize backend communication and authentication-related requests.
- `frontend/src/utils/applyCmsOverrides.js` and related utilities apply CMS-provided content and normalize display URLs.

Backend

- `backend/src/server.js` is the normal backend process entry point.
- `backend/src/app.js` configures Express middleware, security, rate limits, static uploads, health check, route mounts, 404 handling, and centralized error handling.
- `backend/src/routes` maps HTTP endpoints to controllers.
- `backend/src/controllers` implements authentication, CRUD, uploads, page management, audits, recycle bin behavior, and domain operations.
- `backend/src/models` contains domain models and generic SQL-backed model infrastructure.
- `backend/src/config/db.js` owns the MySQL pool and creates required tables for a fresh database.
- `backend/src/middleware` contains authentication, role authorization, and error handling.

3. Curated file tree

This tree intentionally omits `node_modules`, build output, most images/documents, lockfile detail, and one-off maintenance scripts. The repository currently contains about 421 tracked/unignored files: about 269 under the frontend and 141 under the backend.

```

ceypetco-redesign/
├── app.js # Plesk repository-root backend entry
├── package.json # Root workspace/host package (backend)
├── README.md # Project setup summary
├── PLESK_DEPLOYMENT.md # Production deployment instructions
├── frontend/
│   ├── package.json # React/Vite dependencies and scripts
│   ├── vite.config.js # React + Tailwind Vite plugins
│   ├── vercel.json # Frontend hosting configuration
│   ├── .env.example # VITE_API_BASE_URL only
│   ├── public/
│   │   ├── images/ # Public image assets
│   │   ├── documents/ # Downloadable public documents/PDFs
│   │   └── favicon.svg
│   ├── scripts/ # Data-fetch/support scripts
│   └── src/
│       ├── main.jsx # React bootstrap/providers
│       ├── App.jsx # Public site and page composition
│       ├── index.css # Global styling/Tailwind usage
│       ├── api.js # API helper used by public pages
│       ├── routes/
│       │   ├── AppRoutes.jsx # Public/admin route definitions
│       │   └── ProtectedRoute.jsx # Login/role gate
│       ├── context/
│       │   └── AuthContext.jsx # Authentication state
│       ├── i18n/
│       │   └── LanguageContext.jsx
│       ├── services/
│       │   ├── api.js # Axios/API configuration
│       │   ├── authService.js
│       │   └── contentService.js
│       ├── components/ # Shared and public feature UI
│       │   ├── ManagedPage.jsx
│       │   ├── PopupNotice.jsx
│       │   ├── HistoricalPricesPage.jsx
│       │   ├── electric-mobility/
│       │   └── marine-bunkering/
│       ├── admin/
│       │   ├── layouts/AdminLayout.jsx
│       │   ├── components/ # Modal, forms, CRUD, pagination, shell
│       │   └── pages/ # Domain management screens
│       │       ├── Dashboard/
│       │       ├── News/
│       │       ├── Notices/
│       │       ├── Tenders/
│       │       ├── Projects/
│       │       ├── Careers/
│       │       ├── FuelPrices/
│       │       ├── FuelStations/
│       │       ├── Pages/
│       │       ├── Media/
│       │       ├── Users/
│       │       └── RecycleBin/
│       ├── utils/ # CMS, URL, snapshot, image helpers
│       ├── data/ # Fuel station/lubricant JSON
│       ├── assets/ # Imported source assets
│       └── images/ # Imported page images
└── backend/
    ├── package.json # Express API dependencies/scripts
    ├── vercel.json # API hosting configuration
    ├── .env.example # Safe environment-variable template
    ├── api/index.js # Serverless adapter/entry
    ├── uploads/
    │   ├── images/ # Runtime image storage
    │   └── docs/ # Runtime document storage
    └── src/
        ├── server.js # API process entry
        ├── app.js # Express application composition
        ├── config/
        └── env.js # Validation and environment helpers

```

```

├── db.js                # MySQL connection pool/bootstrap
├── middleware/
│   ├── authMiddleware.js
│   ├── roleMiddleware.js
│   └── errorMiddleware.js
├── routes/              # Express routers by domain
├── controllers/         # Request/business logic by domain
├── models/              # Models, registry, generic SQL model
├── utils/               # Tokens, storage, validation, migrations
├── data/                # Seed/reference data
└── seed*.js            # Database seed scripts

```

4. Main domains and features

The route/controller/model organization covers authentication and users; news; notices and popups; tenders and tender downloads; projects; careers; supplier resources/sections; annual reports; management team and contacts; home services; services and divisions; mobile apps; history; managed pages and revisions; recycle bin; media uploads; fuel, aviation, and historical prices; price audit; fuel stations; regional offices; contact messages; dashboard metrics; and a Google Drive image proxy.

Admin routes are protected by authentication. Sensitive areas such as users, contact messages, pricing, tender downloads, page revisions, audits, and recycle bin add role restrictions for `super_admin` and/or `admin`.

5. API and data conventions

- API base path: `/api`. Health check: `GET /api/health`.
- Most CMS operations are mounted below `/api/admin/`.
- Axios is used by the frontend. The deployed API origin is supplied at build time through `VITE_API_BASE_URL`.
- MySQL access uses the promise-based `mysql2` client and a shared connection pool.
- Generic CMS records use a `WebsiteDocuments` table with JSON stored in `LONGTEXT`; dedicated tables also exist for contact messages and media assets.
- Local uploads are validated by type and exposed with separate image/document rules. Documents are served as attachments.
- The backend applies Helmet headers, an explicit CORS allow-list, request-body limits, multiple rate limiters, logging, and centralized error handling.

6. Environment variables

Variable names only are listed here; this document contains no secret values.

Application	Variables
Frontend	<code>VITE_API_BASE_URL</code>
Backend	<code>PORT</code> , <code>HOST</code> , <code>NODE_ENV</code> , <code>CLIENT_URL</code> , <code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code> , <code>DB_SSL</code> , <code>DB_SSL_REJECT_UNAUTHORIZED</code> , <code>JWT_SECRET</code> , <code>JWT_EXPIRES_IN</code> , <code>LOCAL_ASSET_BASE_URL</code>

Production validation requires database settings, an HTTPS client origin, an HTTPS asset base URL, a non-placeholder JWT secret of at least 32 characters, and a least-privilege non-root database user.

7. Development and deployment

Task	Location / command
Install backend	<code>cd backend && npm install</code>
Run backend in watch mode	<code>cd backend && npm run dev</code> — default port 5000
Install frontend	<code>cd frontend && npm install</code>
Run frontend	<code>cd frontend && npm run dev</code> — default port 5173
Lint frontend	<code>cd frontend && npm run lint</code> (Oxlint)
Build frontend	<code>cd frontend && npm run build</code> → frontend/dist
Backend production start	<code>cd backend && npm start</code>

The documented Plesk setup deploys the frontend as static files and the backend as a Node.js 22 application. The preferred API application root is `backend` with startup file `src/server.js`. The repository-root `app.js` and root workspace package provide a supported fallback. Vercel configuration files and `backend/api/index.js` are also present.

8. Guidance for ChatGPT

Use this paragraph with your request:

“Treat the attached PDF as architectural context. Before proposing changes, ask for or inspect the exact relevant files because the PDF is a curated snapshot, not the full source. Preserve the split frontend/backend architecture, existing API conventions, role checks, security middleware, and local upload behavior. Do not invent environment values or expose secrets.”

- Use JavaScript/JSX, not TypeScript, unless a migration is explicitly requested.
- Frontend modules use ES module syntax; backend modules use CommonJS.
- Prefer existing shared services, admin components, generic CRUD/controller/model patterns, and URL/security helpers.
- When changing an API domain, check the matching route, controller, model, frontend service/caller, admin page, and authorization behavior together.
- When changing content, determine whether it is hard-coded fallback content, CMS-managed content, or an uploaded asset.
- Never place passwords, database credentials, tokens, or real `.env` contents in prompts, patches, logs, or documentation.
- Verify frontend changes with lint/build and backend changes with focused runtime/API checks where the required database/environment is available.

9. Scope note

This document was generated from package manifests, configuration, application entry points, routes, deployment notes, and the repository file inventory. It describes the current structure at a useful architectural level; it is not a substitute for sharing the specific source files involved in a future task.

No `.env` values, credentials, private keys, database contents, or uploaded user content are included.